

Error Handling in Large Systems

Result and Option, typed error enums, thiserror and anyhow, context propagation, async and FFI boundaries, logging discipline, and error-contract design for production systems

In a system with many layers, error handling stops being about formatting one message and becomes interface design. This chapter treats Rust errors as production contracts: how to keep domain failures, infrastructure failures, retry policy, and operator context distinct, and how to translate between them only where the meaning actually changes.

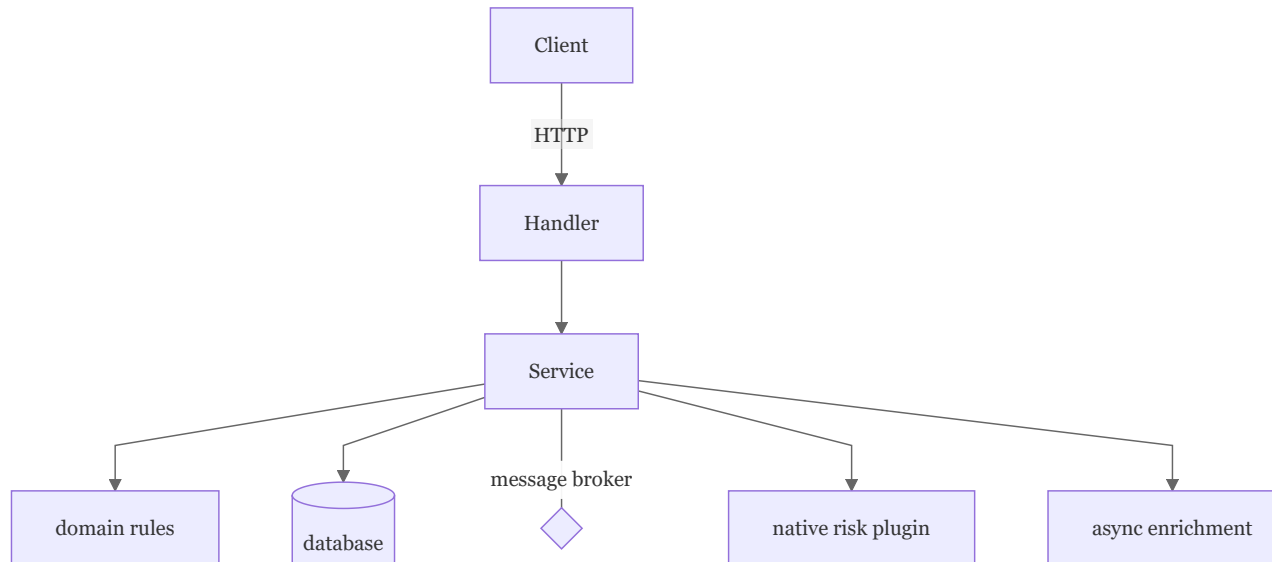
Builds on Chapters 16, 17, 24, 28, 30, and 40

Chapter 16 established domain invariants and domain errors. Chapter 17 introduced Result-first refactoring. Chapter 24 explained async state machines and await boundaries. Chapter 28 covered FFI translation. Chapter 30 covered retries and dead-letter policy at broker boundaries. Chapter 40 is the concrete reminder that even a specialist numeric subsystem fails in ways a caller must act on: a singular matrix or a non-converging solve is a typed Result the caller can fall back from, not a panic, and that is the same contract thinking this chapter applies across every boundary.

Opening scenario

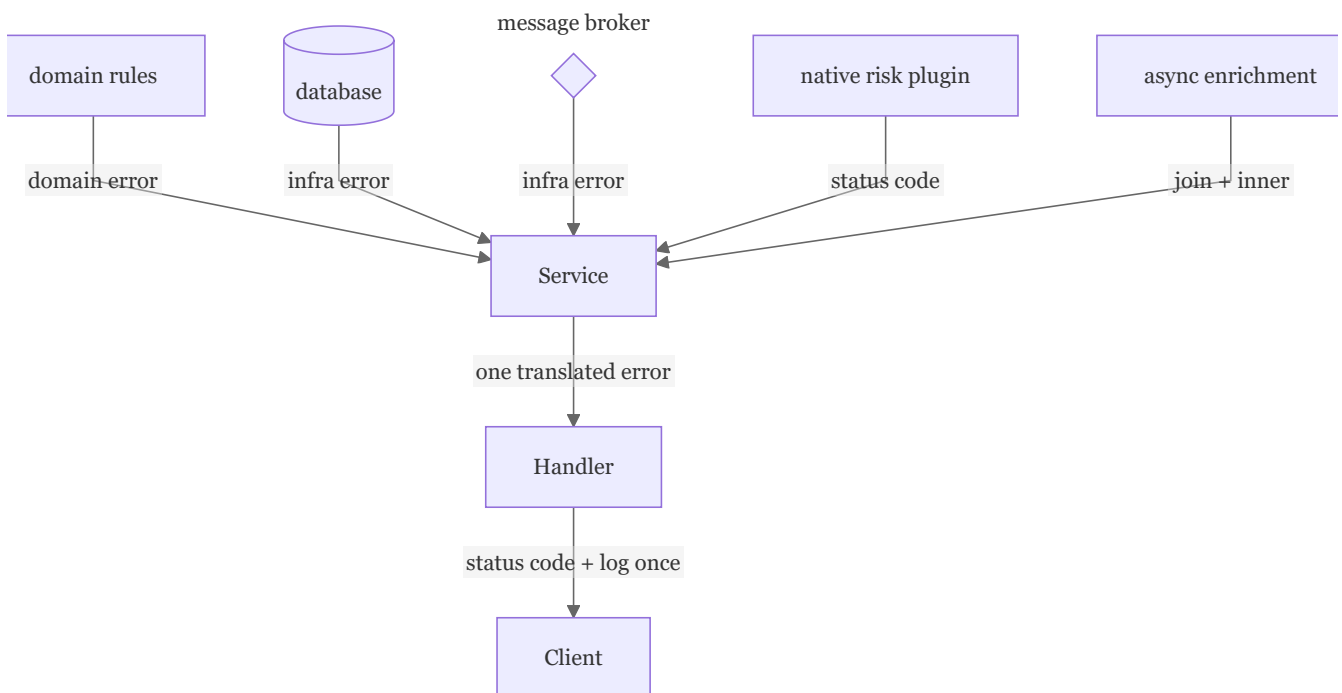
Picture a multi-tenant billing platform. A request arrives over HTTP, the service publishes some internal work to a broker, it calls a native risk-scoring plugin over a C ABI, and it fans out a handful of async enrichment tasks before answering. A single charge can therefore fail in five different vocabularies at once: a business rule can reject it, a database row can be missing, the broker can be unreachable, the native plugin can return a status code, and a spawned task can panic. The job of this chapter is to keep those vocabularies from bleeding into each other.

The requirement the team agrees on is one error policy per boundary. Domain errors stay in domain language and never mention SQL or TCP. Infrastructure errors stay actionable so a caller can retry or dead-letter. Retries are classified explicitly rather than guessed from a message. And every failure is logged exactly once, at the layer where it becomes operationally meaningful, with stable identifiers an operator can correlate later. The diagram below shows where those translation boundaries sit.



Request path: one charge fans out from the service into five dependencies, each speaking its own vocabulary.

Each of those five dependencies can fail differently. The return path collapses those failures back into a single caller-facing contract:



Return path: the service is the single seam where five vocabularies are translated into one caller-facing contract and logged once.

A practical decision order

1. Ask what the caller can still do: retry, reject, translate, or crash.
2. Keep Option local unless absence itself has become a contract violation.
3. Keep typed errors close to the layer that still understands them.
4. Add context at IO, task, queue, and FFI boundaries where meaning changes.

5. Log once, with structure, at the boundary that makes the event operationally relevant.

The in-browser editors simulate output so you can focus on contract shape. In a real Cargo project, the worked examples use ecosystem crates such as `thiserror`, `anyhow`, and `tokio`.

At a glance

- Choose `Option`, `Result`, or `panic` from the caller's next action, not from style preference.
- Keep domain errors, infrastructure errors, and transport-facing errors distinct until the contract truly changes.
- Add context where meaning changes: IO, task joins, queues, FFI, and process boundaries.
- Log once at the boundary that makes the event operationally meaningful.

Caller-action checklist

- Should the caller retry, reject, translate, dead-letter, or crash?
- Does the next layer still need typed classification, or is rich top-level context enough?
- Is this failure local absence, recoverable error, or broken invariant?
- Which stable IDs must travel with the error so an operator can reconstruct the incident later?

Mental model

An error is a contract, not a string to read later

When a function returns a failure, it is making a promise to its caller about what went wrong, what kind of failure it was, and what action still makes sense. In a small program you can get away with a printable message; in a large system that contract is the load-bearing part, because some other layer has to decide whether to retry, reject, translate, or give up. Design the contract first and let the message follow from it, not the other way around.

Let the caller's next move pick the error shape

Before you reach for a type, ask what the code one level up will do with the failure. If absence is ordinary and the caller only needs a local branch, an `Option` is enough. If the caller can retry, map, classify, log with structure, or surface the failure to a user, that is a `Result` with a meaningful error type. A panic is reserved for the cases where there is no sensible next move at all because a program invariant has been violated.

Each boundary wants the error shaped differently

An async task, a message broker, a C ABI, an HTTP handler, and a storage adapter each expect failures in their own vocabulary. The calmest large systems do not force one universal error type through all of them; they let every layer keep an error type it can reason about and translate only at the boundary where the contract genuinely changes. Translation is a deliberate act, performed once, at a named seam.

How to think about this coming from another language

Most senior engineers do not arrive at Rust error handling with a blank slate; they arrive with an exception habit, a runtime habit, or an err-string habit. The shift that matters is not which crate to import. It is where failure becomes visible and who is responsible for deciding what happens next. Find your background below and read the trap as much as the rule.

C++ background

Stop reaching for exceptions and the invisible unwind path as your normal failure channel. In Rust the failure type is part of the function signature, so whether a call can fail, what kind of failure it is, and who is responsible for handling it are all visible at the call site rather than discovered when something throws three frames away. The trap is treating a Result like a checked exception you must immediately rethrow; instead, design the error type so the caller has a real decision to make.

C# background

Drop the instinct toward one ambient exception hierarchy that everything inherits from. A rejected business rule, a request timeout, and a background task that panicked are three different operational events, and Rust rewards keeping them as distinct typed variants instead of collapsing them into a shared base class. The mental shift is that you classify failures at design time, in the type, rather than at catch time, in a chain of catch blocks.

Go background

You already make failure explicit in return values, so the half-step is comfortable. The thing that is different is how far Rust pushes typed classification: rather than accumulating wrapped strings that the next layer has to substring-match, a senior Rust codebase keeps a small enum close to the layer that can still act on the failure. Think less `err != nil` with a formatted message, more a variant the caller can pattern-match and route.

Python background

There is no implicit propagate-until-something-catches behavior at the HTTP edge, and no convention of catching a broad Exception near the top. Fallibility is in the type, and the question driving everything is what the caller can still do: retry, reject with a 4xx, dead-letter, or surface a 5xx. The trap is wanting one giant error type that mirrors a broad except; Rust prefers narrow typed errors near the domain and one rich wrapper only at the very top.

Core concepts

Choosing between Option, Result, and a typed enum

These three are not interchangeable styles; they encode three different statements about a failure. An Option says a value may simply be absent and that absence is unremarkable. A Result says the operation can fail in a way the caller is expected to handle. A typed error enum goes one step further and says the caller may need to tell those failures apart. The common repair in real code is to keep an Option local and promote it into a typed Result at exactly the boundary where missing data stops being routine and becomes a contract violation.

Result and Option

Option models ordinary absence. Result models actionable failure. The common repair is to keep Option local, then promote it to Result at the boundary where missing data stops being routine and starts being a contract violation.

Error enums

Typed enums keep failures reviewable. They let you distinguish invalid state, not found, permission denied, timeout, and retry exhaustion without forcing every caller to parse log text or inspect strings.

A reliable large-system rule is simple: if the caller needs classification, use a typed error. If the caller only needs one local branch on absence, Option is often enough. Promote absence into a typed Result exactly where the contract becomes meaningful.

When to reach for thiserror and when to reach for anyhow

These two crates are not competitors; they answer different questions and most production systems use both. The deciding factor is whether a downstream layer still needs to branch on the error. If it does, you want a named enum with named variants, and thiserror generates the tedious Display and From boilerplate while keeping the structure intact. If the only remaining job is to attach context and surface or log the failure, a single opaque anyhow::Error with a rich context chain is lighter and just as honest.

thiserror

Use thiserror for typed error contracts in libraries, domains, adapters, and public seams. It keeps Display implementations readable while preserving structured variants another layer can still match on or translate.

anyhow

Use anyhow for top-level application orchestration, binaries, workers, CLI entry points, and jobs where rich context is more important than downstream pattern matching. It is usually a boundary aggregation tool, not a domain-model replacement.

This is the usual operational split: typed enums when a downstream layer still branches on the error, rich context wrappers when the boundary is already top-level orchestration and the next action is mostly logging, surfacing, or exiting.

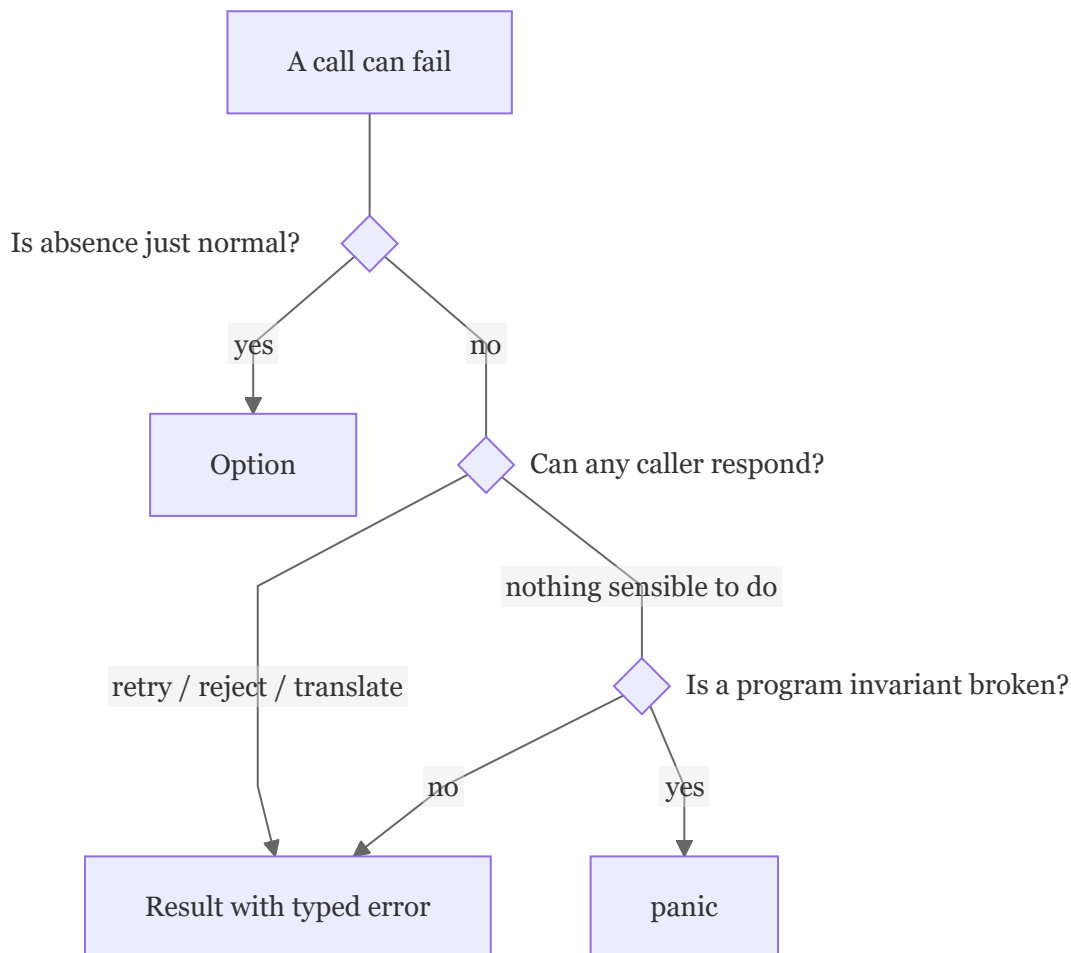
Adding context without burying the source

Context is the answer to the question an operator will ask at 3 a.m.: what was the system trying to do when this failed? A low-level error like "file not found" is true but useless on its own; wrapped with "loading worker config" it becomes diagnosable. The discipline is to add context at the moments where meaning changes and to add it once, never re-stating the same low-level text at every frame on the way up.

- Context should explain the operation that failed, not restate the same low-level error text at every layer.
- Prefer context at boundary crossings: opening a file, fetching from a store, waiting on a task, decoding a message, calling an FFI function, or publishing to a broker.
- Add IDs and resource names in context only when they help operators correlate the failure later.
- Preserve the original source error. Context should add meaning, not erase the chain.

Recoverable versus unrecoverable failures

The dividing line is not how serious the failure feels; it is whether any caller, anywhere up the stack, has a sensible response. Bad input, a rejected business action, or one flaky dependency all have responses, so they belong in a `Result`. A violated invariant has no sensible response because the program is now in a state it was designed never to reach, and that is the narrow place a panic is correct. The decision usually flows like this.



Route by the caller's options, not by how alarming the failure sounds. Panic is the dead end reserved for broken invariants.

Recoverable errors

Bad input, missing configuration, a rejected domain action, one temporary dependency failure, or a timed-out queue claim are all ordinary failures. They belong in `Result` or an equivalent explicit contract the caller can branch on.

Unrecoverable errors

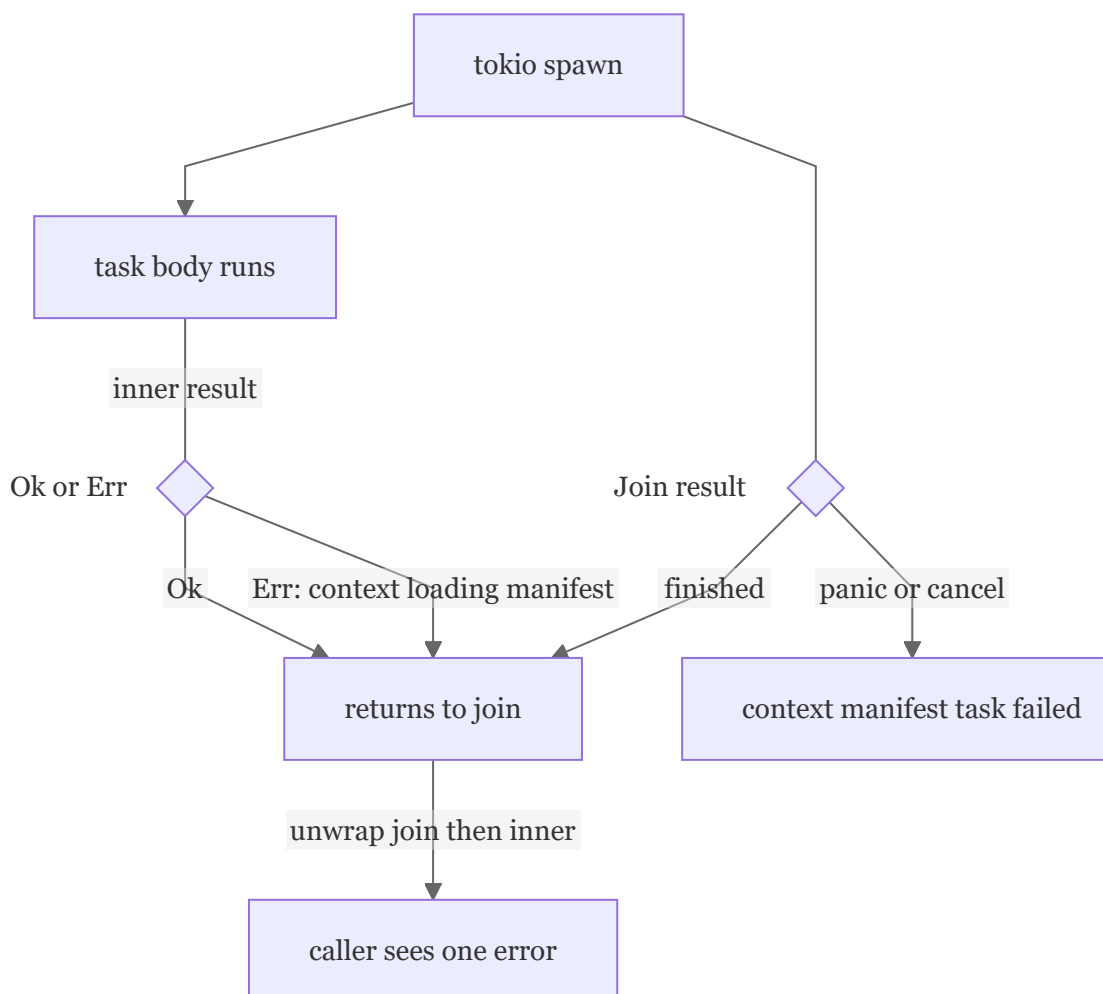
Use `panic` for bugs, broken invariants, and states the process cannot safely continue from: impossible enum states, internal corruption, or violated unsafe preconditions. A panic is not a user-facing validation strategy.

A panic is not Rust's version of a normal business exception. It is the language saying the program's own assumptions are broken badly enough that continuing may be wrong.

Errors that cross an async task boundary

When you spawn a task, you create two stacked failure layers, and a lot of bugs come from forgetting the outer one. The inner layer is whatever the task body returns: an IO error, a domain rejection, an `anyhow::Error`. The outer layer is the join result, which tells you whether the task even finished normally or instead panicked or was cancelled. A panicked task is a different operational event from a missing file, and collapsing them into one string hides exactly the distinction an operator needs. The snippet below is worth reading carefully: notice the two separate `.context(...)` calls and the double `??` at the end, which unwraps the join layer first and then the inner layer.

- A spawned task often creates two error layers: the task join result and the inner task result.
- Add context both before and after await points when the boundary changes meaning. `loading manifest` and `manifest task failed` are different operational events.
- If an error crosses a multithread runtime task boundary, the owned error value usually needs `Send + Sync + 'static`. That constraint is design feedback, not decoration.
- Cancellation and shutdown are not 'success with no value'. Decide whether the caller should see cancellation distinctly from ordinary failure.



A spawned task has a join layer and an inner layer; the double ?? unwraps both, and each gets its own context.

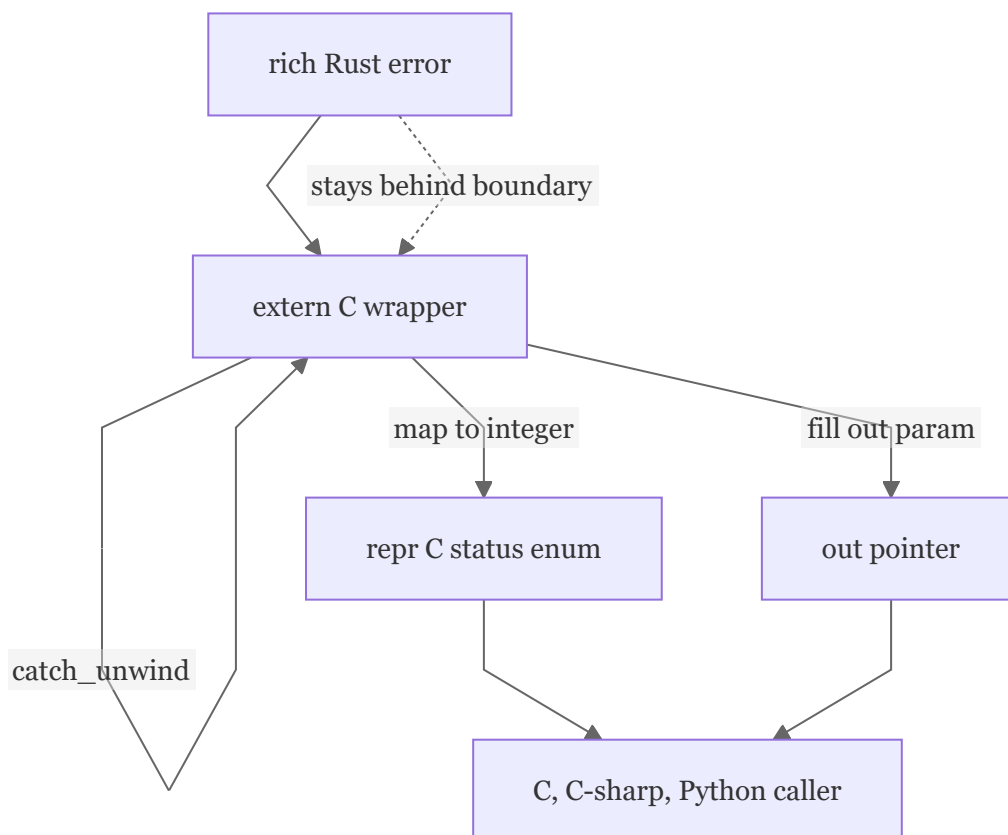
The key lesson is that async failures usually have a task layer and an operation layer. Handle both deliberately instead of collapsing them into one generic timeout or one generic string.

Errors that cross a C ABI boundary

A C ABI cannot carry a Rust enum, an `anyhow::Error`, or the unwinding that a panic implies; letting any of those escape across the boundary is undefined behavior, not a missed opportunity for richness. So the boundary becomes a deliberate narrowing: the rich error stays on the Rust side, and the foreign caller receives the smallest contract it can actually honor, usually a small `#[repr(C)]` status enum plus out-parameters. In the code below, look at the Status enum: each variant is an integer the other runtime can switch on, and the detailed Rust reason never leaves the wrapper.

One requirement is not optional: wrap the inner work in `std::panic::catch_unwind` and map a caught panic to a generic status code. If any inner function can panic, letting that unwind escape across the C ABI is undefined behavior, so the wrapper must catch it at the boundary and turn it into an ordinary status. The diagram shows that catch step explicitly.

- Do not leak Rust enums, `anyhow::Error`, or panic semantics directly across a C ABI boundary.
- Translate to status codes, explicit out parameters, and documented ownership rules.
- Keep the rich Rust error inside the safe side of the wrapper. The foreign side gets the smallest contract it can actually honor.
- If the boundary is public or long-lived, treat the status space itself like a versioned contract.



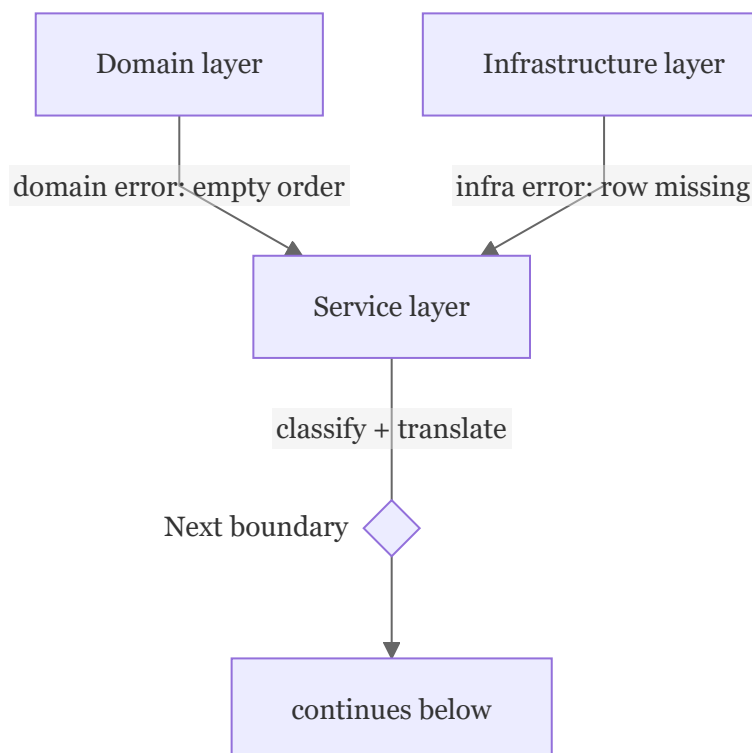
The detailed error stays on the Rust side; the foreign caller sees only a status code and out-parameters.

This is deliberately boring. That is a virtue. The richer Rust-side error chain should stay behind the wrapper where another Rust layer can still use it safely.

The `#[unsafe(no_mangle)]` attribute-wrapper syntax assumes the Rust 2024 edition (rustc 1.82 or newer). On older toolchains, write the bare `#[no_mangle]` attribute instead.

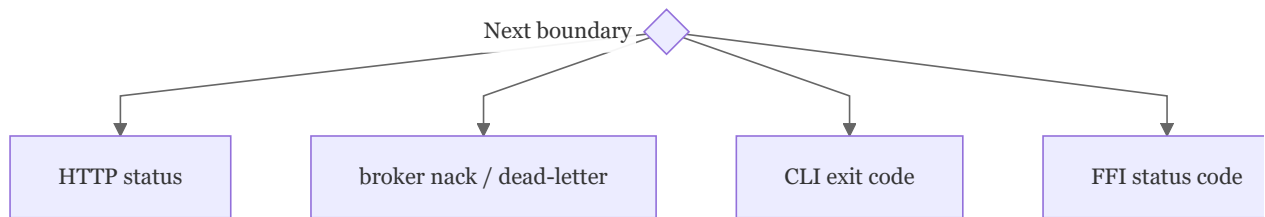
Keeping domain errors and infrastructure errors apart

This is the single distinction that keeps a large error model from rotting. A domain error speaks the language of the business: an order is empty, a transition is invalid, a quota is exhausted. An infrastructure error speaks the language of the machinery: a row is missing, a connection was refused, a task join failed. The two stay separate as they travel up, and the service layer is the one place that combines them and decides how the next boundary should see the result, whether that is an HTTP status, a broker nack, or a CLI exit code. The layering looks like this.



Domain and infrastructure errors stay distinct until the service layer, where classification becomes operational policy.

From that single classify-and-translate point, the service shapes the failure for whichever boundary comes next:



The one classification point fans out into a per-boundary contract: HTTP status, broker nack, CLI exit code, or FFI status code.

Domain errors

Domain errors use domain language: empty order, invalid transition, insufficient quota, duplicate command, unsupported state. They should not mention SQL, HTTP, AMQP, or TCP.

Infrastructure errors

Infrastructure errors describe failed boundaries: timeout, missing row, connection refused, task join failed, queue unavailable, serialization failure. They are not business rules even if the business feels the effect.

Application or service errors

A service layer often combines both, then chooses translation for the next boundary: HTTP status, broker nack, CLI exit code, or FFI status code. This is where classification becomes operational policy.

Logging a failure once, at the right layer

The most common logging mistake in a layered system is not too little logging; it is the same failure logged five times on its way up, each line slightly reworded, until the incident channel is a hall of mirrors. The cure is to pick the one layer where the failure becomes operationally meaningful, log it there with structured fields, and let every layer below simply return the error. Severity should track the caller's reality too: a rejected business command is not an error-level event just because it came back as a Result.

- Log once at the boundary that makes the event operationally relevant. Avoid logging the same failure at five layers on the way back up.
- Include operation name, stable IDs, attempt number, queue or resource name, and classification when those fields help incident response.
- Do not log expected domain rejection at error severity just because it returned Result. A rejected business command may be info, warn, or a client-visible response instead.
- Prefer structured fields over string-only logs when retries, queues, or distributed tracing are involved.
- If you return the error to another layer that will log it with more context, avoid eager log-and-return duplication.

Designing the error contract you expose to callers

A public error contract is an API surface, and it ages like one. The discipline is to expose only what the caller can act on, to keep the retryable-versus-terminal distinction visible somewhere other than a substring of a message, and to resist the catch-all error bag that means nothing in particular. When the contract has to stay stable across teams or languages, translate your internal crates and variants at the

boundary rather than leaking them, because once another runtime depends on a variant you can no longer freely rename it.

- Expose only the error detail the caller can act on. Internal storage-driver text is rarely a good public API contract.
- Keep retryable versus terminal failure explicit somewhere visible. The caller should not infer it from a substring match in one message.
- Prefer small enums over giant catch-all error bags at layer boundaries. If a variant exists, it should mean something specific.
- If the public contract must stay stable across teams or languages, translate internal crates and variants at the boundary instead of leaking them.

Production patterns

- Start from caller action: retry, reject, dead-letter, map to status code, or crash. Then shape the error contract to support that action directly.
- Use typed enums for domain and infrastructure boundaries, then aggregate with anyhow or another richer wrapper only at the top-level orchestration layer when pattern matching stops being useful.
- Add context at IO, task, queue, and process boundaries. That is where a low-level failure becomes an operational event with a name.
- Treat `JoinHandle<Result<T, E>>` and FFI status translation as explicit two-layer error boundaries, not as incidental mechanics.
- Log once with enough structure to debug the incident later, and keep stable IDs flowing through the error path from request or task submission to durable completion.

Pitfalls and tradeoffs

- Returning `anyhow::Error` from deep domain code because it felt faster. That usually erases caller action and leaves the next layer matching on text.
- Using `panic` for malformed input, missing rows, timeout, or other ordinary failure just because another language used exceptions there.
- Logging every error at every layer, then drowning the incident channel in five copies of the same failure with slightly different wording.
- Ignoring the join layer in async task failure and only handling the inner `Result`. A panicked or cancelled task is not the same event as a typed business or adapter error.
- Exporting Rust-shaped errors over FFI or public contracts and discovering too late that another runtime cannot safely consume or stabilize them.

The hardest production mistake is often not one missing match arm. It is a failure contract that never said which layer owned retry, which layer owned translation, and which layer should have logged the incident only once.

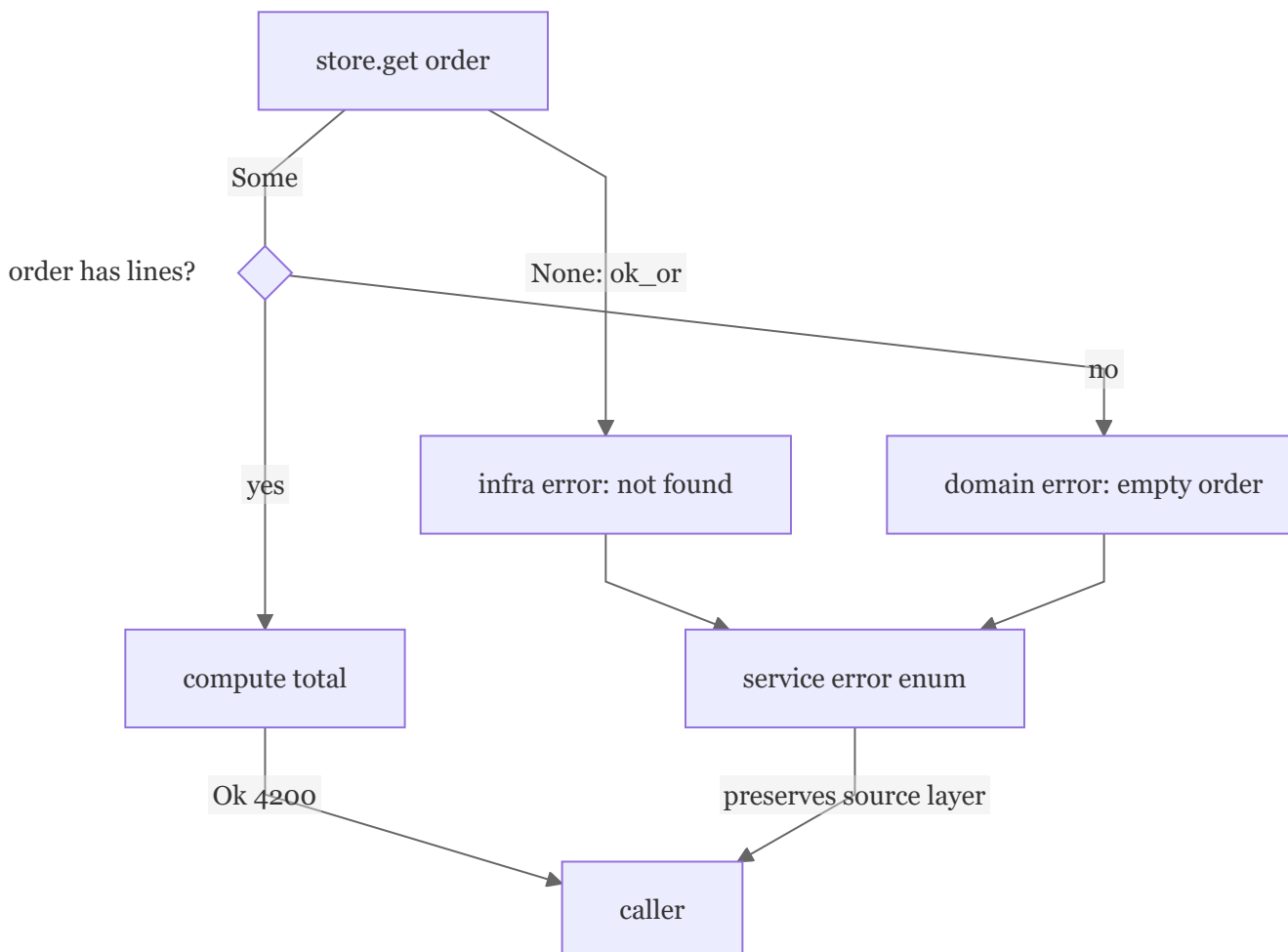
Examples

Both editors simulate their output so you can focus on the shape of the contract rather than on toolchain setup. Read the short pointer and the diagram above each listing first, then run the code and try changing the inputs to see how the error path reshapes.

Example 1: separate domain and infrastructure errors with typed contracts

The service boundary preserves domain language for invalid business state and infrastructure language for missing storage state, then translates both into one service-facing error enum.

What to look at: trace one charge through the function. The store lookup starts life as an Option, gets promoted into an infrastructure error when the row is missing, while the empty-order check raises a domain error in business language. Both reach the service layer, which folds them into a single service-facing enum without erasing which layer actually failed. The flow is below; the code follows.



Absence becomes an infra error; an invalid order becomes a domain error; the service enum carries both without losing which layer failed.

Example 41-1. domain_and_infrastructure_errors.rs — Typed contracts

```

use std::collections::HashMap;
use thiserror::Error;

```

```

#[derive(Debug, Clone)]
struct Order {
    id: &'static str,
    lines: Vec<u64>,
}

#[derive(Debug, Error)]
enum DomainError {
    #[error("order {order_id} has no lines")]
    EmptyOrder { order_id: &'static str },
}

#[derive(Debug, Error)]
enum InfraError {
    #[error("order {order_id} not found in store")]
    MissingOrder { order_id: &'static str },
}

#[derive(Debug, Error)]
enum ServiceError {
    #[error(transparent)]
    Domain(#[from] DomainError),
    #[error(transparent)]
    Infra(#[from] InfraError),
}

fn load_order<'a>(
    store: &'a HashMap<&'static str, Order>,
    order_id: &'static str,
) -> Result<&'a Order, InfraError> {
    store
        .get(order_id)
        .ok_or(InfraError::MissingOrder { order_id })
}

fn total_cents(order: &Order) -> Result<u64, DomainError> {
    if order.lines.is_empty() {
        return Err(DomainError::EmptyOrder {
            order_id: order.id,
        });
    }

    Ok(order.lines.iter().copied().sum())
}

fn bill_order(
    store: &HashMap<&'static str, Order>,
    order_id: &'static str,
) -> Result<u64, ServiceError> {
    let order = load_order(store, order_id)?;
    Ok(total_cents(order))
}

fn main() {
    let mut store = HashMap::new();

```

```

    store.insert(
        "ord-ok",
        Order {
            id: "ord-ok",
            lines: vec![1200_u64, 3000],
        },
    );
    store.insert(
        "ord-empty",
        Order {
            id: "ord-empty",
            lines: vec![],
        },
    );

    println!("ok = {}", bill_order(&store, "ord-ok").unwrap());
    println!("missing = {}", bill_order(&store, "ord-missing").unwrap_err());
    println!("empty = {}", bill_order(&store, "ord-empty").unwrap_err());
}

```

OUTPUT

```

ok = 4200
missing = order ord-missing not found in store
empty = order ord-empty has no lines

```

The store lookup is locally an `Option`, then promoted into a typed infrastructure error at the service boundary.

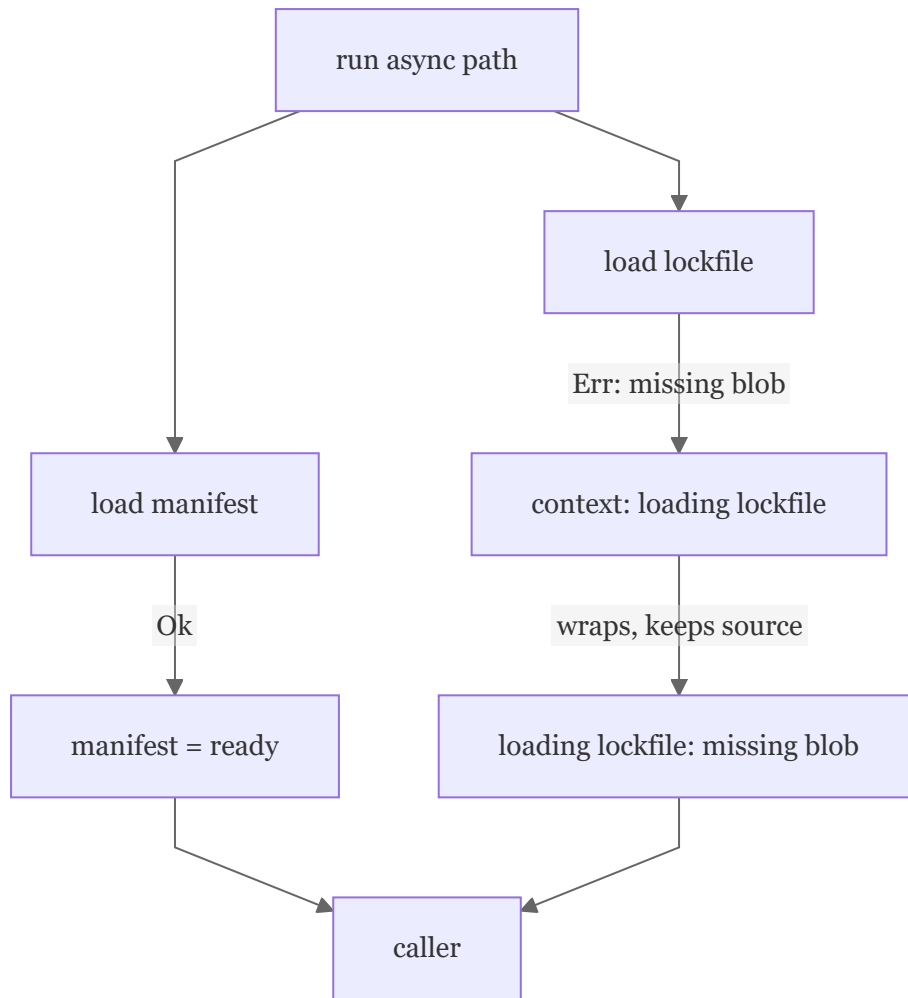
Domain and infrastructure failures stay distinct enough that the next layer can still map or retry them honestly.

The caller sees one service error type, but the error chain still preserves which layer actually failed.

Example 2: add context across async boundaries without losing the source error

The top-level async path uses `anyhow` for orchestration context, while the underlying IO error still survives inside the chain. The spawned task boundary is handled explicitly.

What to look at: there are two paths through this code. The manifest path succeeds and returns `ready`. The lockfile path fails at the IO layer with missing blob, and the `.context("loading lockfile")` call wraps it so the final message reads as a chain rather than a bare cause. Notice the original IO reason is still visible at the tail; context adds meaning, it does not replace the source.



The success path returns a value; the failure path attaches operation context while the underlying IO reason survives in the chain.

Example 41-2. `async_context_propagation.rs` — Async context

```

use anyhow::{Context, Result};
use std::io::{self, ErrorKind};

async fn read_artifact(name: &'static str) -> Result<&'static str> {
    match name {
        "manifest" => Ok("ready"),
        _ => Err(io::Error::new(ErrorKind::NotFound, "missing blob").into()),
    }
}

#[tokio::main]
async fn main() -> Result<()> {
    let manifest = tokio::spawn(async {
        read_artifact("manifest")
            .await
            .context("loading manifest")
    })
    .await
    .context("manifest task failed")??;
}

```

```

    let failed = read_artifact("lockfile")
        .await
        .context("loading lockfile")
        .err()
        .map(|err| format!("{:?}", err))
        .unwrap_or_else(|| String::from("none"));

    println!("manifest = {}", manifest);
    println!("failed = {}", failed);

    Ok(())
}

```

OUTPUT

```

manifest = ready
failed = loading lockfile: missing blob

```

The application shell can keep one rich error channel because the next action is mostly context, logging, and exit or translation.

The join layer and the inner operation layer are both visible, so a panicked task would not be confused with a missing artifact.

The context tells operators which operation failed without replacing the underlying IO reason.

The repository also includes standalone Rust source under `examples/ch41_error_handling_in_large_systems/` including a small FFI status-boundary example alongside the two in-browser worked examples.

Summary

- Option is for local absence; Result is for actionable failure; panic is for broken invariants or unrecoverable process state.
- Typed error enums keep domain and infrastructure failures distinct enough for callers to react honestly.
- `thiserror` is usually the right tool for typed boundary contracts; anyhow is usually the right tool for top-level orchestration context.
- Async task joins, FFI wrappers, brokers, and HTTP APIs are translation boundaries that deserve explicit error contracts.
- Good large-system error handling logs with structure, preserves source context, and exposes only the contract the next layer can actually use.